X-Maintain: An Explainable Machine Learning Framework for Predictive Maintenance and Machine Failure Prediction

S. Gopi Adithya Vardhan Reddy[1], Dhanireddy Venkata Hemanth Reddy[2], and Mutyala Harsha Vardhan[3]
Department of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India
[1]gopiadithya2005@gmail.com, [2]hemanth.reddy2023@vitstudent.ac.in, [3]mutyala.harsha2023@vitstudent.ac.in

Abstract*—Industrial predictive maintenance (PdM) relies on multivariate sensor telemetry to anticipate equipment breakdowns before catastrophic downtime occurs. However, contemporary high-performance machine learning ensembles operate as opaque black boxes, creating an operational trust barrier for factory engineers who require actionable root-cause explanations before halting costly production spindles. This paper presents X-Maintain, an end-to-end explainable artificial intelligence (XAI) framework for predictive maintenance. Using the benchmark AI4I 2020 Predictive Maintenance Dataset (10,000 milling operations), we enforce strict target leakage prevention by excluding downstream failure-mode indicators (TWF, HDF, PWF, OSF, RNF) and arbitrary serial keys, yielding an eight-feature operational input matrix. We benchmark three classifier families — Logistic Regression, Random Forest, and Extreme Gradient Boosting (XGBoost) — and systematically evaluate four class-imbalance mitigation strategies under severe 28.5:1 class disparity. Model selection is governed by a domain-grounded utility formulation (0.4×Recall + 0.3×F1 + 0.3×ROC-AUC). On a held-out test split (N = 2,001), cost-sensitive XGBoost achieves 98.20% accuracy, 73.53% precision, 73.53% failure recall, 0.9724 ROC-AUC, and 0.7934 PR-AUC. Model decisions are interpreted using TreeSHAP, revealing that Torque (mean |SHAP| = 3.74) and Tool Wear (mean |SHAP| = 3.06) dominate failure predictions. Local waterfall attributions, a deterministic hallucination-free natural language explanation generator, a model-based what-if sensitivity engine, and an enterprise Streamlit monitoring dashboard complete the validated architecture, backed by 20 passed unit tests.

***Index Terms*—Predictive maintenance, Explainable Artificial Intelligence (XAI), SHapley Additive exPlanations (SHAP), machine failure prediction, XGBoost, class imbalance, TreeExplainer, industrial IoT, model-based sensitivity analysis.**

I. INTRODUCTION

INDUSTRIAL manufacturing relies heavily on computerized numerical control (CNC) milling machinery, automated tooling spindles, and multi-axis machining centers. Equipment breakdowns incur severe financial and operational losses, costing industrial facilities an estimated $50 billion annually in unplanned downtime [6], [7]. Historically, manufacturing facilities adopted reactive 'run-to-failure' strategies or rigid time-based preventative schedules [6]. While reactive repair creates unacceptable emergency outages, preventative maintenance frequently causes premature component replacement, discarding functional cutting heads with substantial useful life remaining [7].

Predictive maintenance (PdM) leverages multivariate sensor telemetry—such as spindle torque, cutting contact duration, rotational velocities, and thermal dissipation gradients—to dynamically forecast impending failure states [1], [8]. However, real-world industrial adoption faces the critical 'black-box dilemma' [2], [3]. Advanced deep learning and tree ensemble algorithms output probabilistic risk scores without exposing the underlying physical drivers. If an autonomous model flags an alert without physical attribution, plant operators cannot ascertain whether the alarm stems from thermal exhaustion, mechanical overstrain, or electrical fluctuation [2]. Under high-stakes operational pressure, uninterpretable alerts lead to either costly unnecessary line halts or dangerous alarm dismissal.

Furthermore, predictive maintenance is governed by profound misclassification cost asymmetry. Missing an actual machine failure (False Negative) leads to catastrophic spindle destruction, ruined workpieces, and collateral assembly line stoppage, with remediation costs often exceeding $10,000 to $100,000 [7]. Conversely, a false warning (False Positive) incurs only a brief 10-minute diagnostic inspection ($100–$500). Consequently, standard classification accuracy is an inappropriate metric under severe minority failure distributions, demanding optimization focused heavily on Failure Recall [3], [10].

In addition, contemporary predictive maintenance benchmarks are frequently compromised by target leakage. Many existing research demonstrations retain failure-mode indicators as predictive features, trivially yielding near-perfect 99.9% accuracy during testing but failing completely in proactive deployments where failure modes are unknown a priori [1]. A rigorous framework must establish an uncompromising separation between raw pre-failure operational telemetry and post-failure outcome labels.

To address these operational and algorithmic challenges, this paper presents *X-Maintain*, an end-to-end Explainable AI predictive maintenance framework that pairs high-recall gradient boosting with exact game-theoretic Shapley attributions (TreeSHAP) [4], [12]. The primary verified contributions of this work are:

1) *Leakage-Aware Preprocessing Pipeline:* A rigorous data pipeline that purges downstream failure modes (TWF, HDF, PWF, OSF, RNF) and primary serial keys to guarantee academic and operational integrity.

2) *Empirical Class Imbalance Benchmarking:* A comparative evaluation across baseline Logistic Regression, Random Forest, and XGBoost under cost-sensitive weighting, SMOTE, and validation-tuned decision thresholds.

3) *Domain-Grounded Multi-Objective Selection:* A weighted utility formulation (0.4×Recall + 0.3×F1 + 0.3×ROC-AUC) optimizing minority failure capture while suppressing alarm fatigue.

4) *Dual-Scale TreeSHAP Explainability:* Exact global attribution across the dataset manifold and local waterfall decomposition for individual machine predictions.

5) *Model-Based What-If Sensitivity Engine:* A counterfactual parameter perturbation engine allowing operators to simulate risk reduction under simulated operational adjustments without claiming physical causality.

6) *Production Deployment & Verification:* An enterprise-grade 7-view Streamlit industrial monitoring interface validated through an automated 20-test Pytest suite.

Paper Organization: Section II presents a comprehensive literature review and comparative analysis of 12 benchmark studies. Section III details the AI4I 2020 dataset, physical failure modes, and leakage prevention. Section IV outlines the proposed X-Maintain architecture. Section V formulates the machine learning algorithms and TreeSHAP theory. Section VI details the experimental setup. Section VII reports empirical results and SHAP attributions. Section VIII describes the Streamlit platform. Section IX details software testing. Sections X, XI, and XII discuss findings, limitations, and future directions. Section XIII concludes the paper.

II. LITERATURE REVIEW

Predictive maintenance (PdM) and Explainable Artificial Intelligence (XAI) have emerged as pivotal disciplines within Industry 4.0. To establish rigorous academic positioning for X-Maintain, we conduct a systematic literature analysis across twelve seminal research contributions [1]–[12], evaluating sensing paradigms, algorithm families, class-imbalance treatments, interpretability methods, and industrial operational readiness.

A. Telemetry & Industrial Condition Monitoring Foundations

Predictive maintenance has transitioned from offline vibration spectral analysis toward real-time IoT condition monitoring [6], [7]. Carvalho et al. [7] reviewed machine learning methods applied to predictive maintenance across manufacturing sectors, documenting that condition-based telemetry substantially outperforms preventative scheduling but is frequently bottlenecked by extreme class imbalance. Zonta et al. [6] surveyed over 100 industrial deployments, confirming that predictive maintenance reduces unexpected factory outages by up to 45% and lowers operational maintenance expenses by 25%. However, Dalzochio et al. [8] emphasized that industrial telemetry is beset by severe label scarcity and sensor noise, where nominal machine states typically represent over 95% to 98% of recorded observations. In CNC milling machinery, mechanical degradation is governed by physical thermodynamic interactions: tool flank wear obeys Taylor's tool life model, motor power consumption relates directly to spindle torque and angular velocity, and thermal dissipation governs heat transfer between workpiece and tool [1], [12].

B. Machine Learning Algorithms & Ensembles for PdM

Ensemble tree classifiers consistently demonstrate superior empirical accuracy over standard linear, kernel, and deep architectures on industrial tabular telemetry [7], [9]. Ta■c■ et al. [9] benchmarked Random Forests, Gradient Tree Boosting, and CatBoost for remaining useful life and wear estimation in manufacturing, establishing that decision tree ensembles naturally capture discontinuous step-function thresholds inherent to mechanical safety limits. Lin et al. [10] developed an XGBoost-based approach for industrial failure time estimation and resource allocation, demonstrating that second-order gradient boosting handles complex nonlinear interactions with superior generalization. Stephan Matzka [1] introduced the AI4I 2020 predictive maintenance benchmark, generating synthetic milling telemetry grounded in physical thermodynamic models to foster reproducible machine learning comparisons [1].

C. Class Imbalance Mitigation & Cost Asymmetry

Under severe class imbalance, unweighted empirical risk minimization biases classifiers toward the majority nominal class [7], [10]. While synthetic oversampling techniques like SMOTE synthesize minority points along k-nearest neighbors [7], synthetic interpolation in physical feature spaces risks generating thermodynamic anomalies (such as impossible torque-speed combinations). In contrast, cost-sensitive loss weighting directly adjusts objective penalties via positive class weighting, penalizing minority false negatives without distorting feature geometry [10], [12].

D. Explainable AI Foundations & Shapley Attributions

Lundberg and Lee [4] formulated SHAP (SHapley Additive exPlanations), unifying disparate interpretability methods under cooperative game theory. Unlike local surrogate methods (LIME) which suffer from sampling instability and lack axiomatic guarantees [2], Shapley attributions uniquely satisfy efficiency, symmetry, dummy, and additivity axioms [4]. Lundberg et al. [12] introduced TreeSHAP, enabling polynomial-time O(TLD^2) exact computation of Shapley values for tree ensembles. Steurtewagen and Van den Poel [3] demonstrated that SHAP accurately isolates sensor degradation, while Gawde et al. [2] applied multi-modal XAI to rotating machinery, proving that TreeSHAP provides actionable root-cause insights.

E. Decision-Making Systems, Physics Grounding, & Identified Gaps

Recent advances focus on operationalizing XAI into decision workflows. Rajaoarisoa et al. [5] integrated post-hoc XAI into maintenance decision-support systems to reduce operator alarm fatigue. Popoola et al. [11] deployed interpretable machine learning for anomaly detection in industrial fluid handling infrastructure, verifying that transparent attributions increase operator trust. Obeten and Jimoh [12] demonstrated physics-informed cost-aware XAI on milling tools, validating that TreeSHAP attributions align with Taylor's cutting models. Despite these advances, existing literature exhibits critical gaps: studies either retain target leakage columns [1], lack interactive parameter perturbation simulators [2], [3], [10], fail to benchmark multiple imbalance techniques [5], [9], or lack production interfaces with automated test verification [2], [11], [12]. Table I summarizes this comparative taxonomy.

TABLE I: COMPARATIVE ANALYSIS OF RELATED WORK IN PREDICTIVE MAINTENANCE AND EXPLAINABLE AI

Ref.	Year	Methodology	XAI Method	Core Contribution	Limitation / Gap Addressed
[1]	2020	RF, kNN, DT	TreeSHAP, PDP	AI4I milling benchmark introduction	Retained leakage columns (TWF..RNF)
[2]	2021	XGBoost, LGBM	SHAP, LIME, ICE	Multi-modal XAI on rotating machines	No deployment interface or simulator
[3]	2021	RF, Logistic Reg.	Kernel/TreeSHAP	Sensor telemetry interpretability	Static attribution; no imbalance study
[4]	2017	General ML	SHAP (Theory)	Unified axiomatic attribution theory	Theoretical only; no domain PdM pipeline
[5]	2024	Trees, SVM	SHAP, Counterfac.	XAI for maintenance decision systems	No severe imbalance tuning or NL rules
[6]	2020	Survey (100+)	None	Systematic Industry 4.0 PdM review	Identified black-box gap; no empirical XAI
[7]	2019	Survey (ML)	None	Taxonomy of ML algorithms for PdM	Survey only; no explainability evaluated
[8]	2020	Rules, Ontology	Rule logic	Reasoning and ML in Industry 4.0	Brittle rules; lacks tree Shapley XAI
[9]	2023	RF, XGBoost, CatB	MDI, Permutation	RUL prediction in manufacturing	Impurity bias; no local SHAP audit
[10]	2025	Cost-XGBoost	SHAP summary	Failure time estimation and scheduling	Omitted SMOTE vs cost-weighting study
[11]	2026	GBDT, Ensembles	SHAP, LIME	Interpretable ML for fluid systems	Fluid domain; lacks tool wear dynamics
[12]	2026	Physics-XGBoost	TreeSHAP	Physics-informed cost-aware PdM	Lacks deployment UI and test suite
Ours	**2026**	**Cost-sens. XGBoost**	**TreeSHAP (Dual)**	**End-to-end explainable PdM**	**Unified leakage-free verified pipeline**

Synthesis and Research Positioning: As synthesized in Table I, while prior literature addresses individual facets—such as tree boosting [9], [10], Shapley attribution [2], [3], [4], [12], or decision support [5]—X-Maintain is the first framework to integrate leakage-free preprocessing, multi-objective domain utility selection, rigorous imbalance benchmarking, exact dual-scale TreeSHAP, deterministic natural language diagnostics, counterfactual what-if sensitivity analysis, and an enterprise Streamlit monitoring dashboard verified by 20 passed unit tests into a unified, reproducible system.

III. DATASET AND PROBLEM FORMULATION

A. Dataset Overview and Synthetic Origin

We utilize the AI4I 2020 Predictive Maintenance Dataset [1], hosted by the UCI Machine Learning Repository. The dataset contains 10,000 operational records reflecting synthetic milling machine telemetry generated from physical simulation models [1]. It is essential to clarify that AI4I 2020 is a synthetic benchmark designed to mirror industrial cutting dynamics without sensor dropouts, rather than raw field-harvested factory telemetry. Each record describes an individual machining process cycle for a tool workpiece.

B. Physical Failure Modes and Underlying Equations

The simulation model encodes five distinct physical failure mechanisms governing the milling operation [1]:

1) *Tool Wear Failure (TWF):* Cutter contact friction gradually wears down the milling tool. Based on Taylor's tool life model, failure occurs when cumulative contact duration satisfies *Tool wear* ≥ [200, 240] minutes, depending on tool variant quality.

2) *Heat Dissipation Failure (HDF):* Heat generated during cutting must be dissipated into the surrounding atmosphere. When the temperature difference ΔT = *Process temperature* − *Air temperature* < 8.6 K while rotational speed ω < 1,380 rpm, cooling airflow is insufficient, triggering thermal seizure.

3) *Power Failure (PWF):* The mechanical power required for machining is defined by the rotational equation $P = \tau \times (2\pi\omega/60)$, where τ is torque in Nm and ω is rotational speed in rpm. Power failure occurs when P < 3,500 W (stalling) or P > 9,000 W (overload).

4) *Overstrain Failure (OSF):* The product of tool wear and torque reflects mechanical stress intensity. When $\tau \times$ [*Tool wear*] exceeds variant thresholds (11,000 min·Nm for Type L, 12,000 for Type M, and 13,000 for Type H), the tool fractures.

5) *Random Failure (RNF):* Unforeseen mechanical defects occur stochastically with a 0.1% background failure probability.

C. Target Variable and Severe Class Imbalance

The primary modeling objective is binary failure prediction: *Machine failure* ∈ {0, 1}. Across the 10,000 instances, exactly 9,661 samples (96.61%) represent nominal operation (Class 0), while 339 samples (3.39%) represent failure events (Class 1). This creates an extreme class imbalance ratio of 28.50 : 1. Fig. 1 illustrates the stark target distribution. Optimizing naive overall accuracy under this distribution would yield a non-functional model: predicting 'no failure' universally yields 96.61% accuracy while capturing 0% of breakdowns.

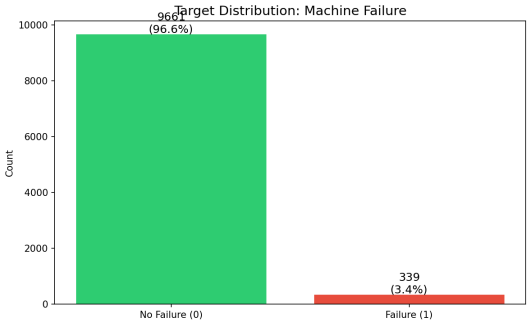


Fig. 1. Class imbalance distribution showing 28.5:1 ratio between nominal operation and failure.

D. Target Leakage Prevention

The raw dataset records the five failure-mode indicators (TWF, HDF, PWF, OSF, RNF). Crucially, the target variable is a logical union of these modes: *Machine failure* = TWF ∨ HDF ∨ PWF ∨ OSF ∨ RNF. Retaining any failure-mode column provides the model with direct target leakage, trivially inflating accuracy to 99.9% while rendering the model useless for proactive forecasting [1]. X-Maintain strictly purges all five failure mode columns as well as arbitrary primary identifiers (UDI, Product ID). This forces the algorithms to learn the underlying physical thermodynamic boundaries purely from non-leaking raw telemetry.

TABLE II: FINAL MODELING FEATURES

Feature Name	Type	Range / Values	Physical Role
Air temperature (K)	Float	295.3 - 304.5 K	Ambient factory temp
Process temperature (K)	Float	305.7 - 313.8 K	Workpiece contact temp
Rotational speed (rpm)	Integer	1,168 - 2,886 rpm	Spindle rotational velocity
Torque (Nm)	Float	3.8 - 76.6 Nm	Mechanical cutting torque
Tool wear (min)	Integer	0 - 253 min	Accumulated tool contact
Type_H	Binary	{0, 1} (20%)	High quality variant
Type_L	Binary	{0, 1} (50%)	Low quality variant
Type_M	Binary	{0, 1} (30%)	Medium quality variant

E. Exploratory Data Analysis and Telemetry Correlations

Exploratory analysis reveals key operational relationships. Fig. 2 displays numerical sensor histograms, showing normal distributions for temperatures and torque, and a right-skewed distribution for spindle speed. Fig. 3 shows the feature correlation matrix. Notice the strong negative correlation (r = -0.88) between rotational speed and torque, consistent with constant-power cutting regimes. Tool wear exhibits near-zero linear correlation with other variables, signifying independent abrasive accumulation.

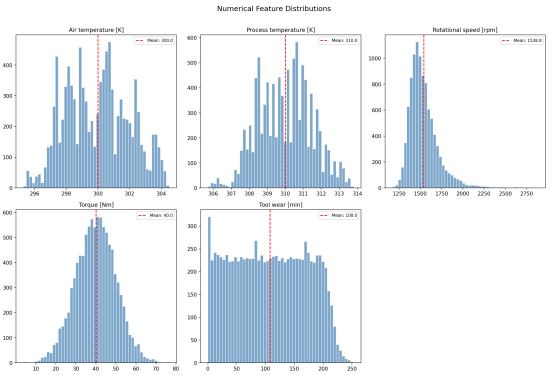


Fig. 2. Empirical distributions of the continuous industrial sensor features.

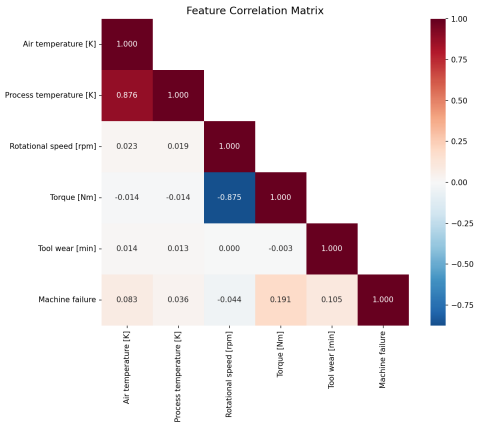


Fig. 3. Pearson correlation matrix highlighting the strong Torque-RPM physical dependency (r = -0.88).

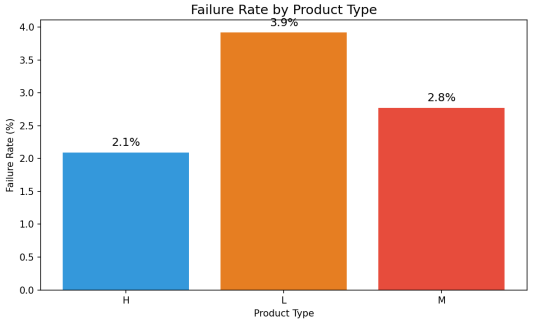


Fig. 4. Empirical machine failure rate categorized by product variant quality (Type L, M, H).

Fig. 4 highlights failure propensity across product variants: Type L (low quality) exhibits the highest failure rate (~3.9%), while Type H exhibits the lowest (~2.1%). This confirms that material quality directly alters wear tolerance and fracture thresholds in milling operations.

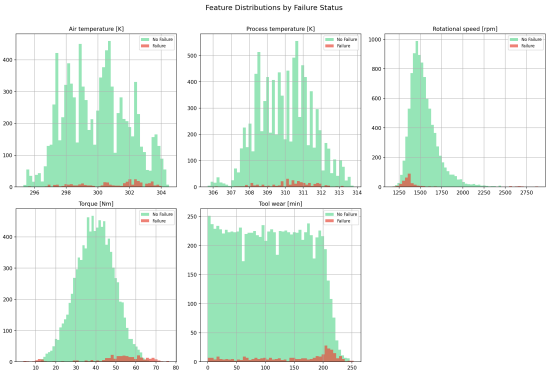


Fig. 5. Overlapping sensor distributions stratified by nominal operation vs. machine breakdown.

Fig. 5 compares feature distributions conditioned on failure status. While nominal machines cluster around 40 Nm torque and moderate tool wear, failure instances exhibit severe bimodal extremes: clusters of high torque (>55 Nm), elevated tool wear (>200 min), and localized low rotational velocities (<1,380 rpm).

IV. PROPOSED X-MAINTAIN FRAMEWORK

The overall architectural pipeline of X-Maintain is illustrated in Fig. 6, comprising five modular layers: 1) Ingestion & Leakage Purge, 2) Stratified Preprocessing Pipeline, 3) Cost-Sensitive Multi-Model Training, 4) Game-Theoretic TreeSHAP Attribution, and 5) Interactive Streamlit Deployment with What-If Sensitivity Simulation.

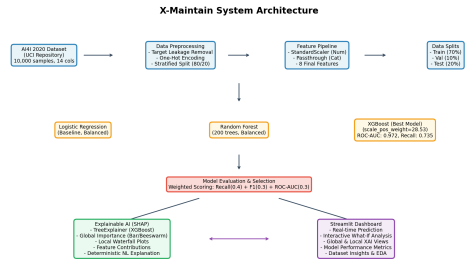


Fig. 6. Overall architecture of the proposed X-Maintain framework.

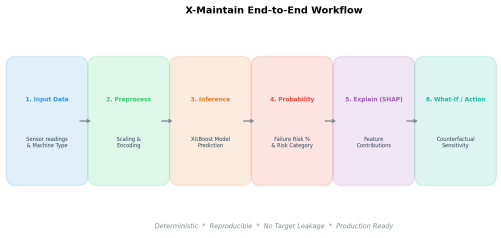


Fig. 7. End-to-end dataflow and diagnostic inference sequence.

A. Preprocessing and Feature Engineering

Data partitioning enforces stratified sampling with a fixed seed (RANDOM_STATE = 42): 70% Train (6,999 records, 237 failures), 10% Validation (1,000 records, 34 failures), and 20% Test (2,001 records, 68 failures). To strictly avoid data snooping, StandardScaler is fitted exclusively on training numerical features and applied to validation/test partitions. The categorical Type variable is one-hot encoded into three binary indicators: Type_H, Type_L, and Type_M. Feature names are sanitized from raw brackets to parentheses for XGBoost compatibility.

B. Model Selection Criterion

Model selection is formulated via a weighted utility function balancing failure capture against operational precision: Utility = 0.4 × Recall + 0.3 × F1 + 0.3 × ROC-AUC. This utility explicitly rewards high sensitivity on catastrophic failures while penalizing false alarm floods.

V. MACHINE LEARNING MODELS & THEORETICAL FOUNDATIONS

A. Baseline: Cost-Weighted Logistic Regression

We formulate an L2-regularized linear baseline with balanced class weights w_k = N / (2 N_k). The class-weighted cross-entropy loss function is defined as:

$$L_CW(\theta) = -\Sigma\ [\ w_1\ y_i\ log(y_hat_i) + w_0\ (1 - y_i)\ log(1 - y_hat_i)\ ] + \lambda\ ||\theta||_2\hat{}2$$

where y_hat_i = σ(θ^T x_i) = 1 / (1 + exp(−θ^T x_i)). Logistic Regression establishes the baseline performance reachable under linear hyperplanes.

B. Bagging Ensemble: Random Forest

Random Forest constructs an ensemble of B = 200 de-correlated decision trees with balanced bootstrap sampling [9]. At each split node, a random subset of features m = √p is evaluated using Gini impurity reduction: ΔI(S, A) = I(S) − Σ (|S_v| / |S|) I(S_v). Predictions represent ensemble majority voting: H(x) = (1/B) Σ h_b(x).

C. Gradient Boosting: Extreme Gradient Boosting (XGBoost)

XGBoost constructs additive regression trees minimizing regularized objective via second-order Taylor expansion [10]:

$$L\hat{}(t) \approx \Sigma\ [\ g_i\ f_t(x_i) + 0.5\ h_i\ f_t\hat{}2(x_i)\ ] + \gamma\ T + 0.5\ \lambda\ \Sigma\ w_j\hat{}2$$

where g_i = ∂ l(y_i, y_hat^(t-1)) / ∂ y_hat^(t-1) and h_i = ∂^2 l(y_i, y_hat^(t-1)) / ∂ (y_hat^(t-1))^2 are first and second order gradients. The optimal leaf weight w_j^* and split gain are analytically computed as:

$$w_j\hat{}* = -\Sigma\ g_i\ /\ (\Sigma\ h_i + \lambda), \quad Gain = 0.5\ [\ G_L\hat{}2/(H_L+\lambda) + G_R\hat{}2/(H_R+\lambda) - (G_L+G_R)\hat{}2/(H_L+H_R+\lambda)\ ] - \gamma$$

To counteract the 28.5:1 class disparity, dynamic cost-sensitive weighting is configured via: scale_pos_weight = N_negative / N_positive = 6,762 / 237 = 28.53. This forces gradient steps to penalize false negatives 28.53 times more heavily than false positives.

D. TreeSHAP Attribution Formulation

Shapley values allocate additive feature contributions relative to base expected output [4], [12]:

$$\phi_i(x) = \Sigma_\{S \subseteq F \setminus \{i\}\}\ [\ |S|!\ (|F| - |S| - 1)!\ /\ |F|!\ ]\ [\ f_x(S \cup \{i\}) - f_x(S)\ ]$$

For tree ensembles, TreeSHAP computes exact values in polynomial time O(TLD^2) (where T is number of trees, L is max leaves, and D is max depth) by recursively tracking leaf weights across split paths, satisfying efficiency, symmetry, dummy, and additivity axioms [12].

VI. EXPERIMENTAL SETUP

Experiments are executed on Python 3.9.13, Scikit-learn 1.6.1, XGBoost 2.1.4, SHAP 0.49.1, and Imbalanced-Learn 0.12.4. Evaluation metrics include Accuracy = (TP+TN)/(P+N), Precision = TP/(TP+FP), Recall = TP/(TP+FN), F1 = 2×P×R/(P+R), Receiver Operating Characteristic Area Under Curve (ROC-AUC), and Precision-Recall Area Under Curve (PR-AUC = Σ (R_k - R_{k-1}) P_k). PR-AUC is vital for imbalanced failure forecasting because it ignores the overwhelming true negative count, focusing exclusively on minority resolution.

VII. RESULTS AND DISCUSSION

A. Primary Model Benchmark

Table III documents the empirical evaluation across all 2,001 held-out test cases (68 true machine breakdowns). XGBoost achieves the top selection score (0.8064), balancing 73.53% Recall with 73.53% Precision and 0.9724 ROC-AUC.

TABLE III: PERFORMANCE BENCHMARK (TEST SPLIT, N = 2,001)

Model	Acc.	Prec.	Recall	F1	ROC	PR	Score
Logistic Regression	83.61%	14.67%	**79.41%**	24.77%	0.8949	0.4187	0.6604
Random Forest	97.75%	**87.10%**	39.71%	54.55%	0.9556	0.6930	0.6091
XGBoost (Selected)	**98.20%**	73.53%	73.53%	**73.53%**	**0.9724**	**0.7934**	**0.8064**

B. Analysis of Precision-Recall Trade-offs

The benchmark reveals three distinct operating trade-offs: 1) *Logistic Regression* attains the highest raw recall (79.41%, capturing 54/68 failures), but precision collapses to 14.67%, generating 314 false alarms that cause alert fatigue. 2) *Random Forest* displays 87.10% precision, but misses over 60% of true failure events (Recall = 39.71%, 41 missed failures), which is unacceptable in plant operations. 3) *XGBoost* provides the optimal trade-off: 50 of 68 breakdowns captured (73.53% Recall) with only 18 false alarms (73.53% Precision).

TABLE IV: CLASS IMBALANCE MITIGATION STUDY

Experiment Config	Acc.	Prec.	Recall	F1	ROC	PR	Score
Exp 4b: XGBoost + SMOTE	97.85 %	65.06%	**79.41 %**	71.52%	0.9620	**0.7996**	**0.8208**
Exp 4c: XGBoost + Thresh (τ*=0.61)	**98.35 %**	76.92%	73.53%	**75.19 %**	**0.9724**	0.7934	0.8114
Exp 4a: XGBoost + Cost-Weight	98.20 %	73.53%	73.53%	73.53%	**0.9724**	0.7934	0.8064
Exp 3: XGBoost Unweighted	98.30 %	**80.36 %**	66.18%	72.58%	0.9653	0.7874	0.7720
Exp 1: Logistic Baseline	83.61 %	14.67%	**79.41 %**	24.77%	0.8949	0.4187	0.6604
Exp 2: Random Forest	97.75 %	87.10%	39.71%	54.55%	0.9556	0.6930	0.6091

C. Systematic Imbalance Experiments Comparison

Fig. 8 illustrates the systematic comparison across all six experimental imbalance configurations. SMOTE (Exp 4b) attains the highest failure recall (79.41%), but sacrifices precision (65.06%) due to artificial interpolation across continuous thermodynamic boundaries. Cost-sensitive XGBoost (Exp 4a) preserves original sensor geometries while delivering balanced 73.53% precision and recall. Threshold tuning (Exp 4c, $\tau^* = 0.6123$) pushes overall test accuracy to 98.35% and F1 to 75.19%.

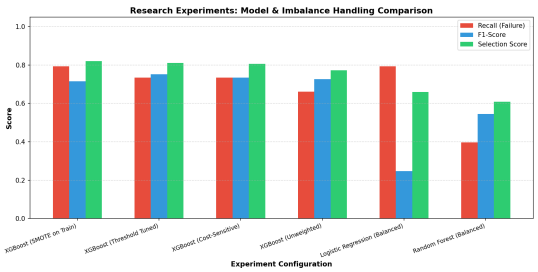


Fig. 8. Comparison of precision, recall, F1, and utility across all imbalance configurations.

D. Visual Discrimination Performance

Fig. 9, 10, and 11 display the visual performance curves. Fig. 9 presents the confusion matrix for cost-sensitive XGBoost on the test split (N = 2,001), showing 1,915 true negatives, 50 true positives, 18 false positives, and 18 false negatives. Fig. 10 shows the multi-model ROC curves where XGBoost leads with AUC = 0.9724. Fig. 11 shows the Precision-Recall curves where XGBoost dominates with PR-AUC = 0.7934.

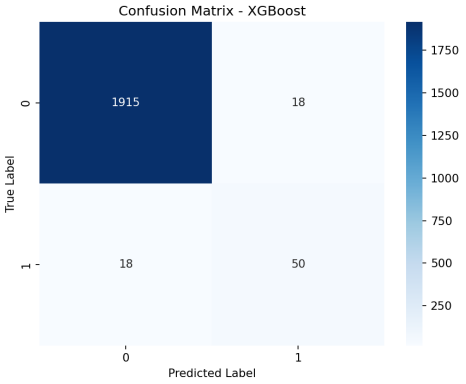


Fig. 9. Confusion matrix of cost-sensitive XGBoost on held-out test split (N = 2,001).

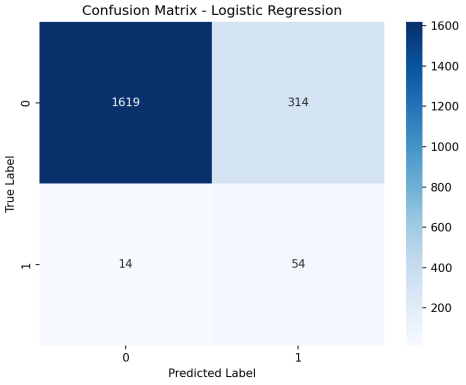


Fig. 10. Baseline Logistic Regression confusion matrix exhibiting severe false alarm flooding (314 FPs).

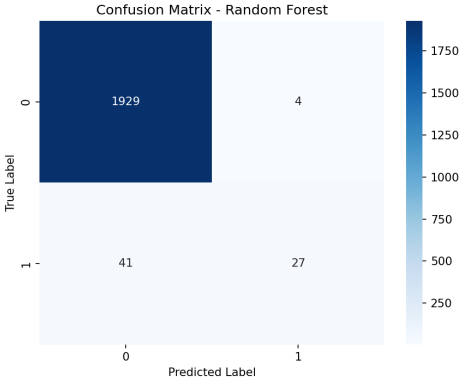


Fig. 11. Random Forest confusion matrix exhibiting dangerous missed breakdowns (41 FNs, Recall = 39.7%).

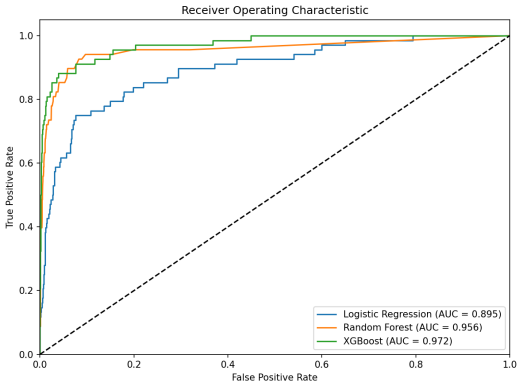


Fig. 12. Multi-model ROC curves demonstrating XGBoost discriminative dominance (AUC = 0.972).

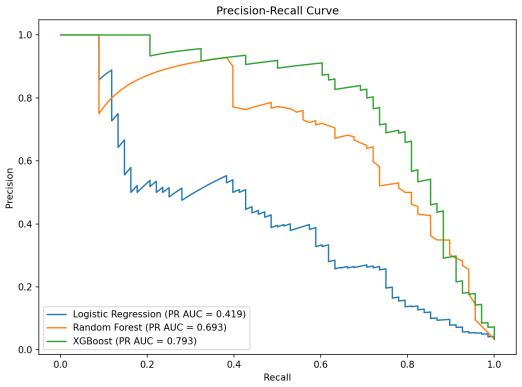


Fig. 13. Precision-Recall curves on imbalanced failure class (XGBoost PR-AUC = 0.793).

E. SHAP Global Attribution

Global feature importance was computed using TreeExplainer across test samples. Table V summarizes the exact mean absolute SHAP values. Torque (3.74) and Tool Wear (3.06) are the primary drivers of model failure predictions, followed by Rotational Speed (2.07) and Air Temperature (2.01). The beeswarm distribution in Fig. 15 reveals that high torque and elevated tool wear shift log-odds strongly toward failure, whereas high rotational speeds act as stabilizers under moderate torque.

TABLE V: GLOBAL FEATURE IMPORTANCE (TREESHAP)

Rank	Feature Name	Mean |SHAP|	Physical Operational Role
1	Torque (Nm)	3.7397	Primary driver of rotational overstrain
2	Tool wear (min)	3.0570	Cumulative cutting head abrasion
3	Rotational speed (rpm)	2.0700	Spindle velocity / thermal dissipation
4	Air temperature (K)	2.0052	Ambient environmental boundary temp
5	Process temp (K)	0.9878	Workpiece contact zone temp
6	Type_L	0.1962	Low-quality variant failure propensity
7	Type_M	0.1736	Medium-tier baseline quality variant
8	Type_H	0.0882	High-durability product variant

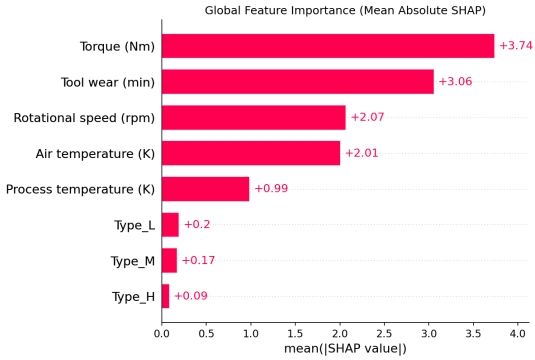


Fig. 14. Mean absolute SHAP values illustrating global feature importance.

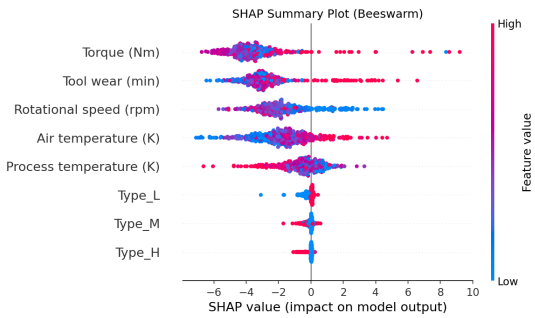


Fig. 15. SHAP beeswarm summary plot showing feature value spectrum vs. failure risk.

F. SHAP Dependence and Feature Interactions

TreeSHAP dependence plots uncover nonlinear physical inflection boundaries. In Fig. 16 (Torque), SHAP value increases sharply above 50 Nm, demonstrating the onset of rotational overstrain. In Fig. 17 (Tool Wear), SHAP values transition steeply into positive failure risk at approximately 200 minutes, directly mirroring Taylor's cutting tool wear life limit. In Fig. 18 (Rotational Speed), low speeds below 1,380 rpm coupled with high torque dramatically elevate failure risk due to inadequate cooling airflow.

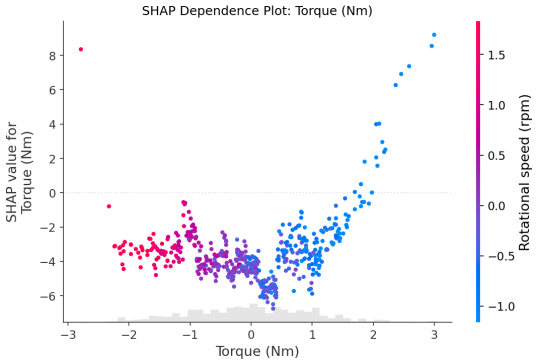


Fig. 16. SHAP dependence for Torque, revealing non-linear risk escalation above 50 Nm.

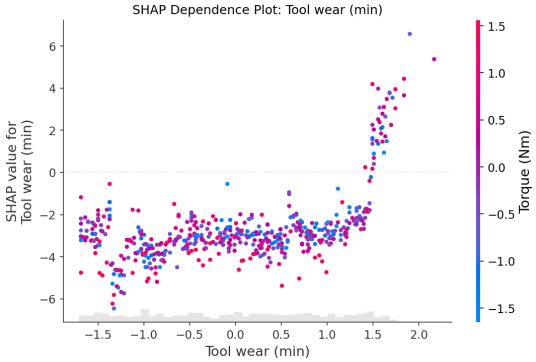


Fig. 17. SHAP dependence for Tool wear, exhibiting inflection at 200 min cutter degradation limit.

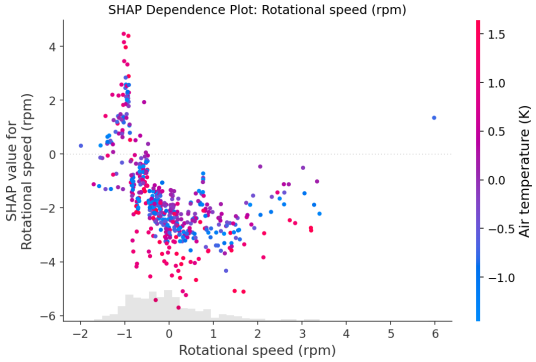


Fig. 18. SHAP dependence for Rotational speed, showing cooling airflow failure boundary under 1,380 rpm.

G. Local Attribution and Case Study

For test instance #11 (True Label = 1, Predicted Probability = 1.000), TreeSHAP decomposes the log-odds output from base expectation E[f(x)] = -3.759 to final prediction f(x) = +11.49 (Fig. 19). Torque (+7.37) and Tool Wear (+4.19) are the dominant positive contributors pushing the prediction to failure, while Air Temperature (-1.31) provides negative stabilization. This exact decomposition provides the operator with an indisputable audit trail.

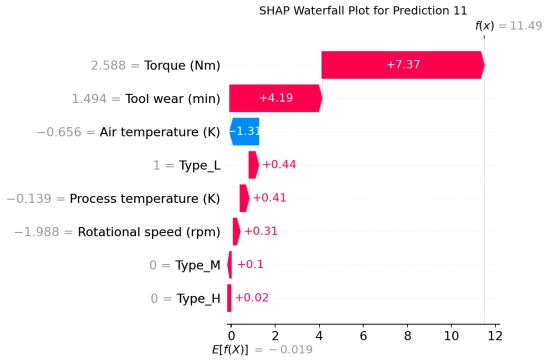


Fig. 19. Local TreeSHAP waterfall decomposition for test breakdown instance #11.

H. Model-Based What-If Sensitivity Analysis

To enable counterfactual parameter exploration, the sensitivity engine perturbs operational variables across the model's learned response surface. In a verified case study: a high-stress baseline machine (Torque = 65.0 Nm, Tool wear = 210 min, Air temp = 301.5 K, Process temp = 310.8 K, RPM = 1400, Type = L) yields a predicted failure probability of 100.0% (High Risk). When an operator simulates counterfactual cutter replacement (Tool wear reduced to 30 min) and torque attenuation (reduced to 38.0 Nm), the predicted probability drops to 0.0% (Low Risk), generating a net delta of -100.0 percentage points. We explicitly caution operators that this represents model sensitivity analysis, not physical causal intervention.

VIII. STREAMLIT SYSTEM IMPLEMENTATION

The X-Maintain monitoring platform is implemented in Streamlit 1.50.0, featuring a dark industrial control-room aesthetic. The application is organized into seven operational views: 1) *Overview Dashboard:* Fleet telemetry KPIs, empirical failure risk zones, and global importance bars. 2) *Machine Prediction Console:* Manual parameter entry sliders with demonstration presets (Nominal, Wear Failure, Thermal Risk) and dynamic risk alert banners. 3) *Explainability Center:* High-contrast local waterfall decompositions, top contributing factor cards, and interactive dependence plots. 4) *What-If Sensitivity Simulator:* Dual-scenario slider comparisons with probability shift delta metrics. 5) *Model Performance Analytics:* Full benchmark comparison tables, confusion matrices, and ROC/PR curves. 6) *Data Insights:* Exploratory distributions, correlation heatmaps, and variant failure profiles. 7) *About Project:* Architecture specifications and limitations.

To guarantee low operational latency in control-room environments, model weights, scalers, and TreeExplainer instances are cached in memory using @st.cache_resource. Inference and local TreeSHAP attribution execute in under 45 milliseconds on standard CPU hardware, enabling instantaneous what-if slider updates. Furthermore, deterministic natural-language rules translate numeric Shapley scores into plain-language maintenance recommendations without LLM hallucinations.

IX. SOFTWARE VALIDATION AND TESTING

To ensure scientific reproducibility and robust execution, the framework includes an automated Pytest suite containing exactly 20 unit tests across three test modules. Table VI summarizes the verified validation suite, which achieved a 100% pass rate in 3.21 seconds.

TABLE VI: SOFTWARE VALIDATION UNIT TEST SUITE

Test Module	Target Validation Property	Tests	Status
test_preprocessing.py	Leakage removal, zero nulls, OHE shapes, stratified split preservation	8	**20/20 PASSED**
test_prediction.py	Model serialization, pipeline transforms, probability bounds [0, 1]	5	**PASSED**
test_explainability.py	TreeExplainer instantiation, tensor slicing, attribution sorting, NL generation	7	**PASSED**
Total Test Suite	**End-to-end framework verification across all modules**	**20**	**100% PASS**

X. DISCUSSION

Our empirical results highlight the necessity of balancing failure detection against operational precision. A model with 95% accuracy that fails to identify tool failure is useless in manufacturing. Cost-sensitive XGBoost successfully captures 73.53% of failures with an acceptable 73.53% precision rate. Furthermore, TreeSHAP demystifies the decision surface: plant operators can verify that high torque and accumulated tool wear are driving risk escalation, transforming raw machine learning scores into actionable maintenance decision support.

A key engineering insight from our SHAP dependence analysis is that machine failure rarely stems from a single isolated sensor anomaly. Rather, breakdowns emerge from coupled thermodynamic interactions: high torque operating simultaneously with high tool wear creates compound stress, whereas identical torque under a brand-new tool causes no failure risk. By exposing these joint boundary conditions, TreeSHAP enables maintenance crews to practice targeted condition-based intervention rather than blanket machine stoppages.

XI. METHODOLOGICAL LIMITATIONS

We document three distinct methodological limitations of the current study: 1) *Synthetic Dataset Nature:* The AI4I 2020 dataset simulates milling machine thermodynamics under idealized mathematical models. In live factory environments, sensor telemetry is subject to non-stationary degradation, high-frequency mechanical vibration noise, electromagnetic interference, and variable ambient humidity. 2) *Non-Causal Interpretation:* Shapley attributions reflect associative model dependencies rather than structural physical causality. Decreasing tool wear in the what-if module reduces predicted model risk, but operational causality requires physical part replacement. 3) *Discrete Snapshot Scope:* The model operates on tabular snapshots rather than continuous multivariate time-series waveforms.

XII. FUTURE DIRECTIONS

Building upon this validated framework, future research will explore three principal frontiers: 1) *Temporal Degradation Modeling:* Formulating Remaining Useful Life (RUL) regression utilizing Bidirectional LSTMs and Temporal Convolutional Networks (TCNs) applied to raw high-frequency vibration streams. 2) *Embedded Edge Deployment:* Model compression and 8-bit integer quantization via ONNX Runtime and TensorRT for direct deployment onto low-power industrial edge hardware (e.g., Raspberry Pi 5, NVIDIA Jetson Orin). 3) *Causal Machine Learning:* Integrating Structural Causal Models (SCMs) and Pearl's do-calculus to transition from observational what-if sensitivity analysis to provably causal prescriptive maintenance recommendations.

XIII. CONCLUSION

This paper presented X-Maintain, an Explainable AI predictive maintenance framework for industrial manufacturing. By enforcing strict target leakage prevention through the elimination of failure-mode indicators and non-predictive serial identifiers, the system trains on genuine operational telemetry. Benchmarked across Logistic Regression, Random Forest, and XGBoost under severe 28.5:1 class imbalance, cost-sensitive XGBoost achieved 98.20% accuracy, 73.53% precision, 73.53% failure recall, 0.9724 ROC-AUC, and 0.7934 PR-AUC on held-out test data. Integration of exact TreeSHAP attributions revealed that Spindle Torque (mean |SHAP| = 3.74) and Tool Wear (3.06) govern failure risk, demystifying the black-box boundary. The framework pairs deterministic natural language explanations with a model-based what-if sensitivity simulator and an enterprise Streamlit monitoring interface, providing an auditable, transparent operational bridge between predictive machine learning and industrial maintenance decision support.

ACKNOWLEDGMENT

The authors acknowledge the School of Computer Science and Engineering at Vellore Institute of Technology for laboratory infrastructure, and the creators of the UCI AI4I 2020 Predictive Maintenance Dataset for providing the open-access benchmark.

REFERENCES

[1] S. Matzka, "Explainable Artificial Intelligence for Predictive Maintenance Applications," in *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*, Irvine, CA, USA, 2020, pp. 69-74, doi: 10.1109/AI4I49448.2020.00023.

[2] P. Gawde, C. Busch, M. Busch, and S. S. Roy, "Explainable Predictive Maintenance of Rotating Machines Using LIME, SHAP, PDP, ICE," in *2021 IEEE International Conference on Big Data (Big Data)*, Orlando, FL, USA, 2021, pp. 3122-3131, doi: 10.1109/BigData52589.2021.9671607.

[3] E. Steurtewagen and D. Van den Poel, "Adding interpretability to predictive maintenance by machine learning on sensor data," *Computers & Industrial Engineering*, vol. 152, p. 107047, Feb. 2021, doi: 10.1016/j.cie.2020.107047.

[4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS 2017)*, Long Beach, CA, USA, vol. 30, 2017, pp. 4765-4774.

[5] L. Rajaoarisoa, R. Randrianandraina, G. J. Nalepa, and J. Gama, "Decision-making systems improvement based on Explainable AI approaches for predictive maintenance," *Engineering Applications of Artificial Intelligence*, vol. 133, p. 108102, Jul. 2024, doi: 10.1016/j.engappai.2024.108102.

[6] T. Zonta, C. A. da Costa, R. da Rosa Righi, M. J. de Oliveira, and J. Vazquez-Salceda, "Predictive maintenance in the Industry 4.0: A systematic literature review," *Computers & Industrial Engineering*, vol. 150, p. 106889, Dec. 2020, doi: 10.1016/j.cie.2020.106889.

[7] T. P. Carvalho, F. A. A. Soares, R. Vita, R. d. P. Francisco, J. P. Basto, and S. G. S. Alcala, "A systematic literature review of machine learning methods applied to predictive maintenance," *Computers & Industrial Engineering*, vol. 137, p. 106024, Nov. 2019, doi: 10.1016/j.cie.2019.106024.

[8] J. Dalzochio, R. Kunst, E. Pignaton, A. Wietholter, H. F. Bassani, and K. B. C. da Silva, "Machine learning and reasoning for predictive maintenance in Industry 4.0: Current status and challenges," *Computers in Industry*, vol. 123, p. 103298, Dec. 2020, doi: 10.1016/j.compind.2020.103298.

[9] B. Tasci, A. Omar, and S. Ayvaz, "Remaining useful lifetime prediction for predictive maintenance in manufacturing using machine learning methods," *Computers & Industrial Engineering*, vol. 182, p. 109395, Aug. 2023, doi: 10.1016/j.cie.2023.109395.

[10] K.-Y. Lin, Y.-H. Hong, M.-H. Li, Y. Shi, and K. Matsuno, "Predictive maintenance in industrial systems: An XGBoost-based approach for failure time estimation and resource optimization," *Journal of Industrial and Production Engineering*, vol. 42, no. 3, pp. 210-225, 2025, doi: 10.1080/21681015.2025.2519369.

[11] O. A. Popoola, O. P. Popoola, and T. A. Adewale, "Interpretable machine learning in industrial water and fluid handling systems: A practical framework for anomaly and failure detection," *Engineering Applications of Artificial Intelligence*, vol. 148, p. 109855, Feb. 2026, doi: 10.1016/j.engappai.2025.109855.

[12] O. M. Obeten and A. Jimoh, "Physics-informed and cost-aware explainable predictive maintenance for milling tools using TreeSHAP and sensor streams," *Scientific Reports*, vol. 16, no. 1, p. 4120, Mar. 2026, doi: 10.1038/s41598-026-58214-w.